Requirements Specification Document

AI-Powered Microservices Health Monitoring Agent

1. Purpose

The purpose of this system is to monitor the health of enterprise microservices and provide an AI-based reasoning layer on top of observability data. The agent will collect health, metrics, logs, and tracing data, analyze service behavior, detect anomalies, identify possible root causes, and recommend remediation actions.

2. Scope

The system will support monitoring for Java Spring Boot microservices deployed in Docker/Kubernetes environments. It will integrate with Prometheus, Grafana, OpenSearch/ELK, Jaeger/Tempo, Kubernetes, and alerting tools such as Slack, Email, Microsoft Teams, or Jira.

The AI reasoning layer may use a local LLM through Ollama or another approved AI model provider.

3. System Objectives

The system shall:

1. Continuously monitor microservice health.
2. Collect metrics, logs, traces, and deployment context.
3. Detect abnormal behavior such as high latency, high error rate, service downtime, memory pressure, and dependency failures.
4. Correlate events across services.
5. Generate human-readable incident analysis.
6. Recommend remediation actions.
7. Notify engineering or operations teams.
8. Support optional human-approved remediation actions.

4. Users and Actors

4.1 Primary Users

- DevOps Engineer
- Site Reliability Engineer
- Backend Developer
- System Administrator
- Support Engineer
- Technical Lead

4.2 External Systems

- Spring Boot Actuator
- Prometheus
- Grafana
- OpenSearch / ELK
- Jaeger / Tempo
- Kubernetes API
- CI/CD pipeline
- Slack / Email / Microsoft Teams
- Jira or incident management system
- Ollama or LLM service

5. Functional Requirements

FR-001: Service Health Collection

The system shall collect health status from each registered microservice using health endpoints such as:

```
/actuator/health
/actuator/info
```

The system shall identify whether each service is:

```
UP
DOWN
DEGRADED
UNKNOWN
```

FR-002: Metrics Collection

The system shall collect service metrics from Prometheus, including:

```
CPU usage
Memory usage
JVM heap usage
GC pause time
Request rate
Error rate
Response latency
Database connection pool usage
Kafka consumer lag
Thread pool usage
```

FR-003: Log Collection

The system shall retrieve and analyze logs from OpenSearch or ELK.

The system shall detect important log patterns, including:

```
Exceptions
Timeouts
Connection refused errors
Database errors
Authentication failures
Retry exhaustion
Circuit breaker open events
OutOfMemoryError
```

FR-004: Trace Collection

The system shall collect distributed tracing data from Jaeger or Tempo.

The system shall identify:

```
Slow request path
Failed downstream dependency
Service-to-service bottleneck
High latency span
External API delay
```

FR-005: Deployment Context Collection

The system shall collect recent deployment information from Kubernetes or CI/CD tools.

The system shall identify whether an incident started after:

```
New deployment
Configuration change
Pod restart
Scaling event
Database migration
Infrastructure change
```

FR-006: Anomaly Detection

The system shall detect anomalies based on configurable thresholds.

Examples:

```
Error rate > 5%
Latency p95 > 2 seconds
Memory usage > 85%
CPU usage > 80%
Service status = DOWN
Database connection pool usage > 90%
Kafka consumer lag increasing continuously
```

FR-007: AI Reasoning Layer

The system shall provide a reasoning layer that analyzes monitoring data and generates explanations.

The reasoning layer shall produce:

```
Affected services
Observed symptoms
Supporting evidence
Likely root cause
Severity level
Recommended remediation
Confidence level
```

FR-008: Incident Classification

The system shall classify incidents using severity levels:

```
LOW
MEDIUM
HIGH
CRITICAL
```

Severity shall be based on business impact, number of affected services, customer impact, and system availability.

FR-009: Root Cause Analysis

The system shall correlate metrics, logs, traces, and deployment history to identify possible root causes.

Example:

```
Payment Service has high error rate.
Order Service latency increased.
Logs show database connection timeout.
Issue started after latest Payment Service deployment.

Likely root cause:
Payment Service database connection pool misconfiguration.
```

FR-010: Recommendation Engine

The system shall recommend remediation steps such as:

```
Restart unhealthy pods
Scale service replicas
Check database connection pool
Rollback latest deployment
Inspect failed dependency
Increase resource limits
Check Kafka consumer lag
Review recent configuration changes
```

FR-011: Alert Notification

The system shall send alerts to configured channels.

Supported channels should include:

```
Slack
Email
Microsoft Teams
Jira
PagerDuty or Opsgenie
```

FR-012: Incident Report Generation

The system shall generate structured incident reports containing:

```
Incident title
Severity
Affected services
Start time
Symptoms
Evidence
```

```
Likely root cause
Recommended actions
Current status
```

FR-013: Human Approval for Actions

The system shall not automatically execute risky production actions without approval.

Actions requiring approval include:

```
Rollback deployment
Restart production pods
Scale production workloads
Change configuration
Trigger failover
```

FR-014: Optional Automated Remediation

The system may support safe automated actions such as:

```
Create Jira ticket
Send alert
Collect diagnostic bundle
Generate incident summary
Trigger read-only Kubernetes inspection
```

6. Non-Functional Requirements

NFR-001: Availability

The monitoring agent shall be highly available and should not become a single point of failure.

NFR-002: Performance

The system shall process monitoring data with minimal delay.

Target:

```
Health check delay: less than 30 seconds
Alert generation: less than 60 seconds after detection
Incident summary generation: less than 2 minutes
```

NFR-003: Scalability

The system shall support monitoring multiple microservices across multiple environments.

Example environments:

```
development
testing
staging
production
```

NFR-004: Security

The system shall secure all API access using authentication and authorization.

The system shall protect:

```
Monitoring data
Logs
Service credentials
Kubernetes access tokens
LLM prompts and responses
Incident reports
```

NFR-005: Auditability

The system shall keep audit logs for:

```
Agent analysis
Generated alerts
Recommended actions
Approved actions
Executed actions
User approvals
```

NFR-006: Explainability

The AI agent shall provide evidence for every recommendation.

The agent shall avoid unsupported assumptions.

NFR-007: Reliability

The system shall continue operating even if one data source is unavailable.

For example, if tracing data is unavailable, the agent should still analyze metrics and logs.

NFR-008: Privacy

Sensitive logs, credentials, tokens, and customer data shall be masked before being sent to the reasoning layer.

7. Data Requirements

The system shall process the following data types:

```
Service health data
Prometheus metrics
Application logs
Distributed traces
Kubernetes pod status
Deployment history
Configuration changes
Alert history
Incident history
```

8. Integration Requirements

The system shall integrate with:

```
Spring Boot Actuator
Prometheus
Grafana
OpenSearch / ELK
Jaeger / Tempo
Kubernetes API
Ollama / local LLM
Slack / Email / Teams
Jira / Incident system
CI/CD pipeline
```

9. AI Prompt Requirements

The reasoning prompt shall instruct the AI agent to:

```
Analyze only the provided data.
Identify unhealthy services.
Use evidence from metrics, logs, traces, and deployment history.
Avoid guessing.
Rank severity.
Recommend practical remediation steps.
Return structured JSON or Markdown output.
```

Example prompt:

```
You are an SRE AI Monitoring Agent.

Analyze the following monitoring data.

Tasks:
1. Identify unhealthy services.
2. Explain symptoms.
3. Correlate metrics, logs, traces, and deployment history.
4. Identify likely root cause.
5. Rank severity.
6. Recommend remediation steps.
7. Provide confidence level.
8. Do not guess beyond the provided evidence.

Monitoring data:
{monitoring_data}
```

10. Output Format Requirement

The system shall generate output in this format:

```
Incident ID:
Severity:
Affected Services:
Current Status:
Symptoms:
Evidence:
Likely Root Cause:
Confidence:
Recommended Actions:
Escalation Required:
```

11. Example Incident Output

```
Incident ID: INC-2026-001

Severity: CRITICAL

Affected Services:
- order-service
- payment-service

Current Status:
Payment Service is degraded. Order Service latency is increasing.

Symptoms:
- Payment Service error rate is 42%
- Order Service p95 latency is 5.2 seconds
- Logs show database connection timeout
- Issue started 8 minutes after latest Payment Service deployment

Likely Root Cause:
Payment Service database connection pool configuration may be incorrect after
the latest deployment.

Confidence:
High

Recommended Actions:
1. Review recent Payment Service deployment.
2. Check database connection pool settings.
3. Roll back Payment Service if configuration changed.
4. Restart affected pods after approval.
5. Monitor error rate and latency after remediation.

Escalation Required:
Yes
```

12. Acceptance Criteria

The system shall be accepted when:

1. It can discover and monitor registered microservices.
2. It can collect health, metrics, logs, and traces.
3. It can detect unhealthy service behavior.
4. It can generate meaningful root cause analysis.
5. It can classify severity correctly.

6. It can send alerts to configured channels.
7. It can generate incident reports.
8. It protects sensitive data before AI analysis.
9. It requires approval before risky production actions.
10. It maintains audit logs of recommendations and actions.

13. Future Enhancements

Future versions may include:

```
Self-healing automation
Predictive failure detection
Capacity planning recommendations
Business impact analysis
Service dependency graph
Historical incident learning
RAG-based troubleshooting knowledge base
ChatOps integration
Voice or chatbot interface
Multi-cloud monitoring
```

14. Recommended Implementation Phases

Phase 1: Basic Monitoring Agent

```
Collect health endpoints
Collect Prometheus metrics
Detect DOWN services
Send alerts
```

Phase 2: Log and Trace Correlation

```
Integrate OpenSearch / ELK
Integrate Jaeger / Tempo
Correlate logs, traces, and metrics
Generate incident summary
```

Phase 3: AI Reasoning Layer

```
Add Ollama or LLM integration
Generate root cause analysis
```

```
Recommend remediation actions
Classify severity
```

Phase 4: Incident Automation

```
Create Jira tickets
Send Slack/Teams alerts
Generate incident reports
Support human approval workflow
```

Phase 5: Advanced Remediation

```
Kubernetes inspection
Controlled pod restart
Scaling recommendation
Rollback recommendation
Self-healing with approval
```